VDF 기본 개념 및 원리

개요

VDF(Verifiable Delay Function)는 Irys 체인의 핵심 합의 메커니즘으로, 시간 지연을 통해 블록 생성 속도를 조절하고 네트워크 보안을 보장하는 암호학적 함수입니다. 이 문서는 VDF의 기본 개념과 Irys에서의 구현 원리를 상세히 설명합니다.

VDF(Verifiable Delay Function)란?

기본 정의

VDF는 다음 세 가지 핵심 속성을 만족하는 암호학적 함수입니다:

1. **순차성(Sequentiality)**: 계산이 순차적으로 이루어져야 하며, 병렬화를 통한 가속이 불가능합니다

2. **검증 가능성(Verifiability)**: 결과를 빠르게 검증할 수 있습니다

3. **유일성(Uniqueness)**: 주어진 입력에 대해 유일한 출력을 생성합니다

수학적 표현

```
VDF(seed, difficulty) → (output, proof)
```

여기서:

- `seed` : 입력 시드 값 (32바이트 해시)
- `difficulty` : 계산 난이도 (반복 횟수)
- `output` : 최종 출력 해시

- `proof` : 검증 가능한 증명 (체크포인트들)

Irys에서 VDF를 사용하는 이유

1. 공정한 블록 생성

전통적인 PoW(Proof of Work)와 달리 VDF는 하드웨어 성능에 의존하지 않고 시간에만 의존합니다. 이는 다음과 같은 장점을 제공합니다:

- **하드웨어 독립성**: 고성능 ASIC이나 GPU가 유리하지 않음
- **예측 가능한 블록 시간**: 일정한 시간 간격으로 블록 생성
- **에너지 효율성**: 불필요한 전력 소모 없음

2. 네트워크 보안

VDF는 다음과 같은 보안 특성을 제공합니다:

```
// 공격자가 미래 블록을 미리 계산하려면
// 정당한 참여자와 동일한 시간이 필요
let  attack_time = legitimate_time; // 병렬화 불가능
```

3. 분산화 촉진

- **진입 장벽 낮음**: 일반 하드웨어로도 참여 가능
- **중앙화 방지**: 특수 하드웨어 불필요
- **공평한 경쟁**: 모든 참여자가 동일한 조건

SHA256 기반 VDF 구현 원리

핵심 알고리즘

Irys는 SHA256 해시 함수를 기반으로 한 VDF를 구현합니다:

```
pub fn vdf_sha(
hasher: &mut Sha256,
salt: &mut U256,
seed: &mut H256,
num_checkpoints: usize,
num_iterations_per_checkpoint: u64,
checkpoints: &mut [H256],
) {
let mut local_salt: [u8; 32] = [0; 32];

for checkpoint_idx in 0..num_checkpoints {
salt.to_little_endian(&mut local_salt);

// 각 체크포인트마다 지정된 횟수만큼 해시 반복
for _ in 0..num_iterations_per_checkpoint {
hasher.update(local_salt);
hasher.update(seed.as_bytes());
*seed = H256(hasher.finalize_reset().into());
}
```

```
// 체크포인트 저장
checkpoints[checkpoint_idx] = *seed;
// 다음 체크포인트를 위한 솔트 증가
*salt = *salt + 1;
}
}
```

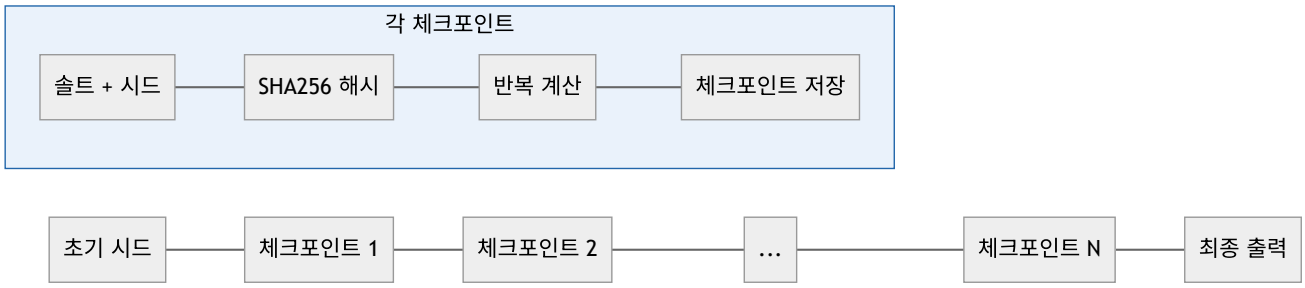
단계별 동작 과정

1. 초기화

```
let mut hasher = Sha256::new();
let mut salt = U256::from(step_number_to_salt_number(config, step_number));
let mut seed = previous_output;
```

2. 체크포인트 계산

각 VDF 스텝은 여러 체크포인트로 구성됩니다:



3. 반복 계산

```
for _ in 0..num_iterations_per_checkpoint {
hasher.update(local_salt); // 솔트 추가
hasher.update(seed.as_bytes()); // 현재 시드 추가
*seed = H256(hasher.finalize_reset().into()); // 새로운 시드 생성
}
```

설정 매개변수

VDF 동작은 다음 매개변수들로 제어됩니다:

```
pub struct VdfConfig {
/// 체크포인트 개수 (기본값: 25)
pub num_checkpoints_in_vdf_step: usize,
/// 1초당 목표 SHA256 연산 횟수
pub sha_1s_difficulty: u64,
/// VDF 리셋 주기 (글로벌 스텝 단위)
pub reset_frequency: usize,
/// 병렬 검증 스레드 수 제한
pub parallel_verification_thread_limit: usize,
}
```

체크포인트당 반복 횟수 계산

```
impl VdfConfig {
```

```rust
pub fn num_iterations_per_checkpoint(&self) -> u64 {
self.sha_1s_difficulty / self.num_checkpoints_in_vdf_step as u64
}
}
```

예시 계산

기본 설정에서의 VDF 계산 예시:

```
설정:
- num_checkpoints_in_vdf_step: 25
- sha_1s_difficulty: 1,000,000
- 체크포인트당 반복 횟수: 1,000,000 / 25 = 40,000

계산 과정:
1. 체크포인트 1: 40,000번 SHA256 해시 → 결과 저장
2. 체크포인트 2: 40,000번 SHA256 해시 → 결과 저장
...
25. 체크포인트 25: 40,000번 SHA256 해시 → 최종 출력

총 계산 시간: 약 1초 (목표)
```

순차성 보장 메커니즘

1. 해시 체인 구조

각 해시 계산은 이전 결과에 의존합니다:

```
// 이전 해시 결과가 다음 계산의 입력이 됨
hash_n+1 = SHA256(salt || hash_n)
```

2. 병렬화 불가능성

```
// 불가능한 병렬 계산 시도
// hash_1000을 계산하려면 hash_999가 필요하고,
// hash_999를 계산하려면 hash_998이 필요하므로
// 모든 중간 단계를 순차적으로 계산해야 함
```

3. 솔트를 통한 예측 불가능성

```
// 각 체크포인트마다 다른 솔트 사용
salt_checkpoint_i = base_salt + i
```

검증 메커니즘

빠른 검증

VDF 결과는 계산보다 훨씬 빠르게 검증할 수 있습니다:

```
pub fn vdf_sha_verification(
```

```
salt: U256,
seed: H256,
num_checkpoints: usize,
num_iterations_per_checkpoint: usize,
) -> Vec<H256> {
// 동일한 알고리즘으로 재계산하여 검증
// 병렬 처리로 검증 속도 향상 가능
}
```

병렬 검증

검증 단계에서는 각 체크포인트를 병렬로 계산할 수 있습니다:

```
let  test: Vec<H256> = pool.install(|| {
(0..config.num_checkpoints_in_vdf_step)
.into_par_iter() // 병렬 반복자 사용
.map(|i| {
// 각 체크포인트를 독립적으로 계산
calculate_checkpoint(i, salt, seed, num_iterations)
})
.collect()
});
```

보안 분석

1. 시간 공격 저항성

```
// 공격자가 더 빠른 하드웨어를 사용해도
// 순차적 특성으로 인해 유의미한 이점 없음
attack_advantage = O(1) // 상수 시간 이점만 가능
```

2. 예측 불가능성

- 솔트와 시드의 조합으로 출력 예측 불가능
- 암호학적 해시 함수의 일방향성 활용

3. 검증 가능성

- 모든 중간 체크포인트 제공으로 검증 가능
- 병렬 검증으로 빠른 유효성 확인

성능 특성

계산 복잡도

- **시간 복잡도**: O(difficulty) - 선형적 증가
- **공간 복잡도**: O(checkpoints) - 체크포인트 수에 비례

검증 복잡도

- **시간 복잡도**: O(difficulty/parallelism) - 병렬화 가능

- **공간 복잡도**: O(checkpoints) - 동일

다음 단계

이 문서에서 다룬 VDF 기본 개념을 바탕으로, 다음 문서들에서는 더 구체적인 구현 세부사항을 다룹니다:

- 체크포인트 시스템 - 체크포인트 생성 및 검증 프로세스
- 난이도 조정 메커니즘 - 동적 난이도 조정 및 리셋 시스템
- 성능 최적화 - VDF 계산 최적화 기법